Task Design Proposal: MLflow Call Completion

**Open to revision*

1. Overview

Each evaluation instance is a masked-call completion task built from a real MLflow Python file in the collected dataset. The model receives the code surrounding one hidden MLflow API call and must produce the missing call. Model output is compared against the original human-written call, giving a direct, per-call measure of how well each model generates MLflow tracking code in realistic contexts.

2. Instance definition

For a file F and a masked call site c:

- Input: a context window of the 40 lines preceding c and the 10 lines following c, with the call line replaced by a mask marker. The file path and repository name are retained as metadata but not shown to the model.
- Target: the original call line or lines, for example mlflow.log_params(safe_params).
- One instance per qualifying call site. A file with 8 direct calls yields up to 8 instances.

3. Design decisions and rationale

3.1 Context: fixed window rather than whole file

A window of 40 lines before and 10 lines after the mask balances realism against cost. Whole-file context is more faithful, but the dataset contains files exceeding 800 lines (for example skrl_training.py in microsoft/physical-ai-toolchain), which would dominate GPU time and exceed practical context budgets at 8B scale. A fixed window also standardizes difficulty across instances. Window size is a tunable hyperparameter and can be revisited after pilot runs.

3.2 Unit: single masked call rather than whole block

Masking one call per instance gives an unambiguous ground truth and clean automatic scoring. Whole-block generation (for example an entire with mlflow.start_run() body) is more realistic but admits many valid answers, making automatic comparison unreliable; it is deferred to a possible second experiment or to the manual rubric evaluation.

3.3 Qualifying call sites: direct mlflow calls only

Instances are generated only from call sites counted by the AST detector: direct call expressions on the bare mlflow module name. The ten-repository manual validation identified four real-world patterns that make broader definitions unreliable: (1) lazy or defensive imports, (2) the module stored as an object attribute (self._mlflow.log_...), (3) local variables shadowing the mlflow name, and (4) MagicMock objects named mlflow in tests. Restricting to direct calls excludes patterns 2 through 4 by construction and keeps ground truth unambiguous. Pattern 1 (lazy imports) does not affect call sites and requires no exclusion.

Additional exclusions: call sites inside test files (paths containing /tests/ or files named test_*.py) are excluded, since test scaffolding is not representative of tracking code; this also removes the mock pattern entirely.

3.4 Prompting: two modes as a comparison axis

Each instance is run in two modes per model:

- Completion mode: the raw code prefix is provided and the model continues it, matching base-model usage.
- Instruction mode: a short instruction wraps the context ("Complete the missing MLflow call at the marker"), matching instruct-model usage.

This makes prompting style an explicit experimental factor rather than a confound, and accommodates the fact that the selected checkpoints differ: Llama 3.1 8B Instruct is instruction-tuned, while the Qwen3-8B checkpoint serves both modes. [OPEN QUESTION for Corey: confirm the final checkpoint choice, base versus instruct, for each model family.]

4. Estimated instance counts

The current pre-widening dataset contains 105 files across 33 repositories with roughly 450 counted direct calls in total. After excluding test files, the estimate is approximately 350 to 450 instances. The widened collection pass now running is expected to increase this substantially. Final counts will be reported when the widened dataset is complete.

5. Scoring

Automatic, per instance:

- Exact match (normalized whitespace).
- AST match: parse both target and output; compare the call name and argument structure.
- CodeBLEU on the generated line or lines against the target.

Aggregate: per model and per prompting mode; precision, recall, and F1 over the API call name; distribution of error types feeding the failure taxonomy. Manual: rubric scoring on a stratified sample, as planned in the experiment design.

6. Pipeline sketch

1. instance_builder.py: reads mlflow_files.csv, fetches each file, locates direct call sites via AST, emits instances.jsonl (context, target, metadata).
2. run_inference.py: SLURM array job; loads one model on one 40GB MIG slice, generates completions for all instances in both modes; writes outputs.jsonl.
3. score.py: computes the metrics above; writes results.csv.

4. Analysis notebook: tables, comparisons, and error clustering for the failure taxonomy.

7. Open questions for supervisors

1. Base versus instruct checkpoints for each model family (affects Section 3.4).
2. Whether whole-block generation should be a second experiment after the single-call results are in.
3. Target instance count if the widened dataset is very large (cap or use all).